FXDE Plugin System 完全設計書

0. 目的

本設計は、FXDE を単体機能アプリではなく、将来的に Pattern / Indicator / Strategy / AI / Overlay / Signal / Risk 拡張を後付けできる **拡張可能プラットフォーム** として成立させるための Plugin System を定義する。

今回の要件として、以下を必須とする。

- 導入済みプラグインが **一覧表示** されること
- 各プラグインは **イメージビュー + 説明 + メタ情報** を持つ **パネルカード形式** で表示されること
- プラグインの **ソース情報は閲覧のみ** とし、その画面から編集不可であること
- 各プラグインは **有効 / 無効** を切り替え可能であること
- FXDE の既存 7 ページ構成と衝突せず、特に **PG-03 Strategy** を中心に自然に統合されること

---

1. 正式な位置づけ

FXDE は 7 ページ構成が正本であり、`/strategy` は PG-03 として既に確定している。Plugin System は新規の独立主要ページではなく、以下のいずれかで統合する。

1.1 推奨統合先

第1候補: PG-03 ****/strategy****** に Plugin Manager セクションを統合**

理由:

- Strategy は現時点で本実装の余地が大きく、拡張機能の管理画面を内包しやすい
- パターン検出・指標分析・シグナル生成・AI補助など、プラグインとの意味的親和性が高い
- 既存ページ構成を壊さずに拡張できる

1.2 ルーティング方針

v5.1 では新規トップレベルページは増やさない。

したがって Plugin UI は以下の構成とする。

- `/strategy`
- `overview` タブ
- `patterns` タブ
- `backtest` タブ
- `plugins` タブ ← 新設

これにより、Part 10 の 7 ページ構成を保持したまま拡張できる。

---

2. Plugin System の設計思想

2.1 何をプラグイン化するか

FXDE　においてプラグイン化の対象になるのは、アプリの根幹インフラではなく　**分析ロジック・描画ロジック・補助ロジック** とする。

プラグイン対象

- Pattern Detection Plugin
- Indicator Plugin
- Strategy Logic Plugin
- Risk Filter Plugin
- Overlay Plugin
- Signal Generator Plugin
- AI Summary Adapter Plugin
- External Connector Adapter Plugin（将来）

プラグイン対象外

- Auth
- Users
- Settings Core
- Role / RBAC Core
- Billing
- Session / JWT
- Prisma Core Models
- Global Error Handling

2.2 Plugin の責務

プラグインは「FXDE コアが提供する標準入力を受け、標準出力に変換して返す拡張モジュール」である。

- コアが相場データ・Snapshot・Indicators・Trades を渡す
- Plugin は独自ロジックを実行する
- 結果を FXDE 標準フォーマットで返す
- UI はその標準フォーマットを描画する

この方式により、個別プラグインごとに UI や DB 契約を崩さず、差し替え可能性を保つ。

---

3. ユーザー要件を満たす UI 設計

3.1 Plugin Manager 一覧画面

表示形式

カードグリッド表示 を標準とする。

各カードには以下を表示する。

1. サムネイル画像
2. プラグイン名
3. プラグイン種別
4. 短い説明
5. バージョン
6. 作者 / 提供元
7. 対応カテゴリタグ
8. 現在状態（有効 / 無効 / エラー / 非互換）
9. 詳細ボタン
10. Enable/Disable トグル

カード例

- 上部: 16:9 の image preview
- 中段: タイトル + 種別バッジ
- 下段: 説明文 2〜4 行
- フッター: version / source / status / toggle

レイアウト仕様

- Desktop: 3〜4 カラム
- Tablet: 2 カラム
- Mobile: 1 カラム

並び順デフォルト

1. 有効プラグイン
2. 推奨プラグイン
3. 無効プラグイン
4. 非互換 / エラー

フィルタ

- All
- Enabled
- Disabled
- Pattern
- Indicator
- Strategy
- AI
- Overlay
- Risk

ソート

- 名前順
- 更新日順
- インストール日順
- 有効状態順
- 推奨順

3.2 プラグインカード詳細項目

各カードが最低限持つフィールドは以下。

- `displayName`
- `pluginType`
- `summary`
- `coverImageUrl`
- `version`
- `authorName`
- `sourceLabel`
- `status`
- `isEnabled`
- `compatibility`
- `capabilities[]`
- `supportedSymbols[]`（任意）
- `supportedTimeframes[]`（任意）

3.3 プラグイン詳細パネル

カードクリック時は右サイドの Drawer または Modal で詳細を表示する。

詳細パネルの構成

1. Hero image
2. プラグイン名 / version / author
3. 長文説明
4. できること（Capabilities）
5. 入力データ
6. 出力データ
7. 実行タイミング
8. 依存プラグイン
9. 権限要求
10. ログ出力有無
11. Source 情報
12. 有効 / 無効トグル

3.4 Source 表示要件

ユーザー要件にある「プラグインのソースが同じように見えること、その画面からの変更不可」を以下のように定義する。

方針

- 各プラグインの実コードは **read-only viewer** で表示する
- エディタ風 UI に見えるが、編集不可

- コピーは可とするかどうかは設定で制御可能
- 保存ボタン・編集ボタン・inline edit は存在しない

具体仕様

Source パネル

- モノスペースフォント
- 行番号表示
- シンタックスハイライト
- `Read Only` バッジ固定表示
- `Edit Disabled in FXDE` 注記表示

禁止事項

- この画面からコード変更不可
- rename 不可
- delete 不可
- save 不可
- upload overwrite 不可

Source 表示対象

実際の完全ソース全文ではなく、原則として以下のどちらかを採用する。

v5.1 推奨

- `sourcePreview`（先頭 N 行 + メタ情報）
- `manifest excerpt`
- `exported interface summary`

将来オプション

- 完全ソース全文の read-only 表示

理由

- セキュリティ
- 知財保護
- 実行中プラグインの不正改変防止
- UI からコード編集させないため

3.5 Enable / Disable 要件

トグル仕様

各カードにトグルスイッチを設置する。

- ON = Enabled
- OFF = Disabled

切り替え時の挙動

Enable

- plugin registry 上で有効化
- capability ごとの hook を runtime に登録
- 必要なら cache warmup
- UI に success toast 表示

Disable

- runtime hook から除外
- スケジュール実行停止
- overlay / signal / strategy 反映停止
- UI に disabled toast 表示

注意

- core 依存の必須 plugin は disable 不可にできる
- 依存関係がある場合、親 plugin 無効化時に確認ダイアログを出す

例:

- 「このプラグインを無効化すると、依存する 2 個のプラグインも停止します」

---

4. UX 詳細

4.1 状態バッジ

各カードの状態は以下のどれか。

- `enabled`
- `disabled`
- `error`
- `incompatible`
- `missing_dependency`
- `update_available`

色

- enabled: 緑
- disabled: グレー
- error: 赤
- incompatible: 黄
- missing_dependency: 橙
- update_available: 青

4.2 空状態

プラグイン未導入時は以下表示。

- イラスト
- 「導入済みプラグインはまだありません」
- 「標準プラグインを追加する」CTA

4.3 トグル制約

以下の場合はトグルを無効化する。

- 権限不足
- 非互換バージョン
- 必須依存不足
- システム保護対象

その場合の UI:

- Disabled switch
- 理由ツールチップ

---

5. ドメインモデル

5.1 PluginManifest

```
export type PluginType =
  | 'pattern'
  | 'indicator'
  | 'strategy'
  | 'risk'
  | 'overlay'
  | 'signal'
  | 'ai'
  | 'connector';

export interface PluginManifest {
  id: string;
  slug: string;
  displayName: string;
  version: string;
  descriptionShort: string;
  descriptionLong: string;
  pluginType: PluginType;
  authorName: string;
  sourceLabel: string;
  homepageUrl?: string;
```

```
  docsUrl?: string;
  coverImageUrl?: string;
  iconUrl?: string;
  readmeMarkdown?: string;
  sourcePreview?: string;
  entryFile: string;
  checksum: string;
  fxdeApiVersion: string;
  fxdeWebVersion: string;
  capabilities: string[];
  permissions: string[];
  dependencies: string[];
  optionalDependencies: string[];
  tags: string[];
  isCore: boolean;
  isSigned: boolean;
  installScope: 'system' | 'user';
}
```

5.2 InstalledPlugin

```
export interface InstalledPlugin {
  id: string;
  pluginId: string;
  installedAt: string;
  installedByUserId: string | null;
  isEnabled: boolean;
  enableUpdatedAt: string;
  status:
    | 'enabled'
    | 'disabled'
    | 'error'
    | 'incompatible'
    | 'missing_dependency'
    | 'update_available';
  errorMessage: string | null;
  lastHealthCheckAt: string | null;
  lastExecutedAt: string | null;
  configLocked: boolean;
}
```

5.3 PluginCardViewModel

```
export interface PluginCardViewModel {
  pluginId: string;
  displayName: string;
  pluginType: PluginType;
  summary: string;
```

```
  coverImageUrl: string | null;
  version: string;
  authorName: string;
  sourceLabel: string;
  isEnabled: boolean;
  status: InstalledPlugin['status'];
  compatibilityLabel: string;
  tags: string[];
  isCore: boolean;
  sourcePreviewAvailable: boolean;
}
```

---

6. DB 設計

v5.1 に導入する場合はコアテーブルを壊さずに追加テーブルで対応する。

6.1 plugin_manifests

- プラグインの定義情報
- manifest の正本

主カラム:

- id
- slug
- display_name
- plugin_type
- version
- description_short
- description_long
- author_name
- source_label
- cover_image_url
- readme_markdown
- source_preview
- entry_file
- checksum
- fxde_api_version
- fxde_web_version
- capabilities_json
- permissions_json
- dependencies_json
- optional_dependencies_json
- tags_json
- is_core
- is_signed
- created_at
- updated_at

6.2 installed_plugins

- 実際に利用中のプラグイン状態

主カラム:

- id
- plugin_manifest_id
- installed_by_user_id
- is_enabled
- status
- error_message
- config_locked
- installed_at
- enable_updated_at
- last_health_check_at
- last_executed_at

6.3 plugin_audit_logs

- enable / disable などの監査ログ

主カラム:

- id
- plugin_manifest_id
- action
- actor_user_id
- before_state_json
- after_state_json
- created_at

6.4 plugin_runtime_cache

- runtime 状態キャッシュ
- Redis でも可

---

7. バックエンド API 設計

7.1 一覧取得

`GET /api/v1/plugins`

役割

- Plugin Manager 一覧表示用

レスポンス

- カード表示に必要な情報一式

7.2 詳細取得

`GET /api/v1/plugins/:pluginId`

役割

- 詳細パネル表示

7.3 source preview 取得

`GET /api/v1/plugins/:pluginId/source-preview`

役割

- read-only source viewer 用

注意

- 書き換え API は作らない
- `PATCH /source` や `PUT /source` は存在しない

7.4 enable

`POST /api/v1/plugins/:pluginId/enable`

7.5 disable

`POST /api/v1/plugins/:pluginId/disable`

7.6 health check

`GET /api/v1/plugins/:pluginId/health`

7.7 audit log

`GET /api/v1/plugins/:pluginId/audit-logs`

---

8. 権限制御

閲覧

- 全ロール閲覧可でもよいが、v5.1 は **全ロール閲覧可** を推奨

有効 / 無効切替

- `ADMIN` のみ変更可、または
- `PRO_PLUS | ADMIN` に許可

推奨方針

v5.1 では安全性優先で以下。

- 一覧閲覧: `FREE | BASIC | PRO | PRO_PLUS | ADMIN`
- 詳細閲覧: `FREE | BASIC | PRO | PRO_PLUS | ADMIN`
- enable / disable: `ADMIN` のみ

これなら分析画面からコード変更できないという要件と整合しやすい。

---

9. ランタイム構造

9.1 Plugin Registry

アプリ起動時に manifest と installed state を読み込み、registry を生成する。

```
export interface RegisteredPlugin {
  manifest: PluginManifest;
  installed: InstalledPlugin;
  moduleRef: unknown | null;
}
```

9.2 Lifecycle

- discover
- validate
- dependency resolve
- register
- enable
- execute
- disable
- unload

9.3 安全策

- checksum 検証
- 署名検証（将来）
- version compatibility check
- capability whitelist
- timeout
- exception isolation

---

10. フロント実装構成

```
apps/web/src/
  pages/
    Strategy.tsx
  components/strategy/plugins/
    PluginManager.tsx
    PluginToolbar.tsx
    PluginGrid.tsx
    PluginCard.tsx
    PluginDetailDrawer.tsx
    PluginSourceViewer.tsx
    PluginStatusBadge.tsx
    PluginEnableToggle.tsx
    PluginEmptyState.tsx
  hooks/
    usePlugins.ts
    usePluginDetail.ts
    usePluginToggle.ts
  lib/
    plugin-formatters.ts
```

---

11. UI コンポーネント詳細

11.1 PluginCard

props

```typescript
interface PluginCardProps {
  plugin: PluginCardViewModel;
  onOpenDetail: (pluginId: string) => void;
  onToggle: (pluginId: string, nextEnabled: boolean) => void;
  canToggle: boolean;
}
```

セクション

- cover image
- title row
- type badge
- summary
- tags
- source label
- version
- status badge
- toggle row

11.2 PluginSourceViewer

仕様

- read-only
- 行番号あり
- syntax highlight
- copy button は任意
- edit button なし

表示文言

- `Source Preview`
- `Read Only`
- `Editing is disabled in FXDE`

11.3 PluginEnableToggle

- ON/OFF switch
- ローディング表示
- 失敗時ロールバック

---

12. 状態遷移

enable

`disabled -> enabling -> enabled`

disable

`enabled -> disabling -> disabled`

failure

`enabling/disabling -> error`

UI では optimistic update ではなく、v5.1 は **server confirmed update** を推奨。

---

13. 非機能要件

性能

- Plugin 一覧初回表示: 1.5s 以内目標
- カード画像 lazy load
- 詳細は on-demand fetch

セキュリティ

- source edit API 不提供
- signed plugin のみ enable 可とする将来拡張余地
- path traversal 防止
- plugin entry path はサーバ管理

監査

- enable/disable は必ず監査ログに記録
- actor / timestamp / before / after を保持

可観測性

- plugin error count
- last executed time
- health status

---

14. 既存 FXDE との結合ポイント

Dashboard

- enabled signal plugins の結果のみ表示

Chart

- enabled overlay plugins のみ描画

Prediction

- enabled strategy/AI plugins のみ補助表示

Settings

- plugin global behavior の保護設定のみ持つ
- 個別 plugin 設定編集は v5.1 では未実装でも可

Strategy

- Plugin Manager のメイン配置先

---

15. v5.1 での実装境界

実装する

- Plugin manifest 読み込み
- 導入済み一覧表示
- カード表示
- 画像プレビュー
- 詳細表示

- read-only source preview
- enable/disable
- audit log
- status badge

実装しない

- 画面上でのソース編集
- plugin marketplace 決済
- 外部公開 plugin store
- sandbox 実行基盤
- plugin code upload from UI
- dynamic live coding

---

16. 推奨 MVP

Phase 1

- DB テーブル追加
- GET /plugins
- GET /plugins/\:id
- POST enable/disable
- Strategy の Plugins タブ
- Card UI
- Source Preview Drawer

Phase 2

- health check
- audit log UI
- dependency warning
- update available badge

Phase 3

- signed plugin
- registry validator
- plugin pack import
- capability-level permission control

---

17. 実装ルール

- ソース変更系 API は作らない
- UI からコード編集不可
- enable/disable は監査必須
- core plugin は誤停止防止
- Strategy ページ配下に統合
- FXDE 既存ロール体系を壊さない
- 既存 7 ページ構成を増やさない

---

18. 最終結論

この Plugin System により、FXDE は

- 単なる固定機能の分析アプリ

から

- **拡張可能な FX 分析プラットフォーム**

へ進化できる。

特に今回の要件である

- 一覧表示
- イメージ付きパネルカード
- ソースの read-only 表示
- その画面から変更不可
- 有効 / 無効切替

は、v5.1 の範囲でも十分実装可能であり、既存仕様とも整合する。

実装上は **PG-03 Strategy 内の Plugins タブ** として入れるのが最も自然で、安全で、FXDE 全体設計とも噛み合う。

---

19. Claudeへ渡す実装プロンプト版

以下は、Claude Code / Claude Desktop の新規会話へそのまま投入できる **実装修正プロンプト完成版** である。

```
# FXDE Plugin System 実装プロンプト

あなたは FXDE プロジェクトの実装担当です。
目的は、既存 FXDE に対して **Plugin System を v5.1 準拠で安全に追加実装** することです。

## 最重要前提

- Plugin System は **新規トップレベルページを追加しない**
- 実装位置は **PG-03 `/strategy` 内の `plugins` タブ** とする
- 既存 7 ページ構成は絶対に壊さない
- UI から plugin source を **編集不可** にする
- plugin の **有効 / 無効切替** は可能にする
- plugin の source 表示は **read-only viewer** に限定する
- enable / disable は **監査ログ必須**
- 既存の Auth / Users / Settings Core / Billing / JWT / Prisma Core を plugin 化しない

## 作業前に必ず確認する正本資料
```

以下を必ず先に読んで、現物コードと仕様を突合してから実装開始すること。

1. `SPEC_v51_part5.md`
2. `SPEC_v51_part10.md`
3. `FXDE_v51_wireframe_integrated.html`
4. `FXDE Plugin System 完全設計書`

必要なら既存の Strategy ページ、Sidebar、ルーティング、RBAC、API 命名規則、packages/types の DTO/Schema の現物も必ず確認すること。

実装目標

以下を実装する。

Backend
- Prisma schema へ plugin 系テーブル追加
- NestJS に `plugins` module 追加
- 一覧 API
- 詳細 API
- source preview API (read-only)
- enable API
- disable API
- audit logs API
- plugin registry service の最小実装

Frontend
- `/strategy` ページに `plugins` タブ追加
- Plugin Manager 一覧 UI
- カードグリッド UI
- 詳細 Drawer / Modal
- read-only source preview viewer
- enable / disable toggle
- filter / sort の基本 UI
- status badge

Types / Contracts
- packages/types に plugin DTO / schema / response contract を追加
- API / Web の双方で型共有すること

実装ルール

1. ****既存のページ構成は増やさない****
2. ****新規トップレベル route を追加しない****
3. ****source 編集 API を作らない****
4. ****UI からコード編集機能を作らない****
5. ****enable / disable は ADMIN のみ変更可**** を初期値とする
6. ****一覧・詳細閲覧は全ロール可**** を基本とする
7. ****server confirmed update**** を採用し、optimistic update は避ける
8. 監査ログには actor / timestamp / before / after を必ず残す
9. DB 命名、DTO 命名、controller/service/module 構成は既存 FXDE 規約に合わせる
10. 既存コードを消さず、差分追加中心で実装する

実装スコープ

今回実装するもの
- plugin_manifests
- installed_plugins
- plugin_audit_logs
- Plugin Registry 最小版
- GET `/api/v1/plugins`
- GET `/api/v1/plugins/:pluginId`
- GET `/api/v1/plugins/:pluginId/source-preview`
- POST `/api/v1/plugins/:pluginId/enable`
- POST `/api/v1/plugins/:pluginId/disable`
- GET `/api/v1/plugins/:pluginId/audit-logs`
- Strategy 内 Plugins タブ
- Card UI
- Detail Drawer
- Source Preview Viewer
- EnableToggle

今回実装しないもの
- plugin marketplace
- plugin upload UI
- plugin source 編集
- live coding
- sandbox 実行基盤
- 決済連動
- 外部 plugin store

期待するファイル追加・変更候補

apps/api
- `src/modules/plugins/plugins.module.ts`
- `src/modules/plugins/plugins.controller.ts`
- `src/modules/plugins/plugins.service.ts`
- `src/modules/plugins/plugins.registry.ts`
- `src/modules/plugins/plugins.repository.ts`（既存構成に合わせて必要なら）
- `src/modules/plugins/dto/*.ts`

packages/types
- `src/schemas/plugin.schema.ts`
- `src/plugin.ts` または既存 export 体系に従うファイル

prisma
- `schema.prisma`
- migration
- 必要なら seed

apps/web
- `src/pages/Strategy.tsx` または既存 Strategy ページ関連
- `src/components/strategy/plugins/PluginManager.tsx`

- `src/components/strategy/plugins/PluginToolbar.tsx`
- `src/components/strategy/plugins/PluginGrid.tsx`
- `src/components/strategy/plugins/PluginCard.tsx`
- `src/components/strategy/plugins/PluginDetailDrawer.tsx`
- `src/components/strategy/plugins/PluginSourceViewer.tsx`
- `src/components/strategy/plugins/PluginStatusBadge.tsx`
- `src/components/strategy/plugins/PluginEnableToggle.tsx`
- `src/hooks/usePlugins.ts`
- `src/hooks/usePluginDetail.ts`
- `src/hooks/usePluginToggle.ts`

データ仕様

PluginManifest
必須フィールド:
- id
- slug
- displayName
- version
- descriptionShort
- descriptionLong
- pluginType
- authorName
- sourceLabel
- coverImageUrl
- sourcePreview
- entryFile
- checksum
- fxdeApiVersion
- fxdeWebVersion
- capabilities[]
- permissions[]
- dependencies[]
- optionalDependencies[]
- tags[]
- isCore
- isSigned
- installScope

InstalledPlugin
必須フィールド:
- id
- pluginManifestId
- installedByUserId
- isEnabled
- status
- errorMessage
- configLocked
- installedAt
- enableUpdatedAt
- lastHealthCheckAt

- lastExecutedAt

status enum
- enabled
- disabled
- error
- incompatible
- missing_dependency
- update_available

UI 要件

一覧画面
- カードグリッド
- 画像 preview
- plugin 名
- 種別 badge
- summary
- version
- author/source
- status badge
- enable/disable toggle
- detail open

詳細画面
- Hero image
- 長文説明
- capabilities
- dependencies
- permissions
- source preview
- read-only バッジ
- toggle

Source 表示
- syntax highlight
- 行番号あり
- 編集不可
- save / edit / rename / delete 不可
- `Read Only` と `Editing is disabled in FXDE` を表示

実装手順

1. 現在の routing / sidebar / strategy page / types export / auth guard / role guard / prisma naming を現物確認
2. Prisma schema 追加
3. packages/types に schema / DTO 追加
4. Nest plugins module 実装
5. controller / service / RBAC 実装
6. mock seed または最低限の初期データ投入
7. Strategy 内 plugins タブ実装

8. API 接続 hooks 実装
9. source preview read-only viewer 実装
10. toggle と audit log 動作確認
11. typecheck / build / lint / run 確認

完了条件

以下を満たしたら完了。

- `/strategy` 内に `plugins` タブが表示される
- plugin cards が表示される
- detail drawer が開く
- source preview が read-only 表示される
- enable/disable が API 経由で切り替わる
- audit log が残る
- 型エラー 0
- 既存 route / sidebar / auth を破壊していない
- 新規トップレベルページを増やしていない

出力ルール

- 変更したファイル一覧
- 主要設計判断
- migration 内容
- API 一覧
- build / typecheck 結果
- 未実装残件

不明点があっても止まらず、まず既存コードの現物確認を優先し、既存規約に合わせて安全に実装を進めること。

---

20. Prisma schema + Nest API + React UI の具体コード仕様版

以下は、実装時の **具体コード仕様** である。

20.1 Prisma schema 仕様

```
model PluginManifest {
  id                  String            @id @default(cuid())
  slug                String            @unique
  displayName         String
  version             String
  descriptionShort    String            @db.VarChar(300)
  descriptionLong     String            @db.Text
  pluginType          PluginType
  authorName          String
  sourceLabel         String
  homepageUrl         String?
```

```
  docsUrl              String?
  coverImageUrl        String?
  iconUrl              String?
  readmeMarkdown       String?            @db.Text
  sourcePreview        String?            @db.Text
  entryFile            String
  checksum             String
  fxdeApiVersion       String
  fxdeWebVersion       String
  capabilitiesJson     Json
  permissionsJson      Json
  dependenciesJson     Json
  optionalDepsJson     Json
  tagsJson             Json
  isCore               Boolean            @default(false)
  isSigned             Boolean            @default(false)
  installScope         PluginInstallScope @default(system)
  createdAt            DateTime           @default(now())
  updatedAt            DateTime           @updatedAt

  installedPlugins     InstalledPlugin[]
  auditLogs            PluginAuditLog[]

  @@map("plugin_manifests")
}

model InstalledPlugin {
  id                 String       @id @default(cuid())
  pluginManifestId   String
  installedByUserId  String?
  isEnabled          Boolean      @default(false)
  status             PluginStatus @default(disabled)
  errorMessage       String?      @db.Text
  configLocked       Boolean      @default(true)
  installedAt        DateTime     @default(now())
  enableUpdatedAt    DateTime     @default(now())
  lastHealthCheckAt  DateTime?
  lastExecutedAt     DateTime?

  pluginManifest     PluginManifest @relation(fields: [pluginManifestId], references: [id],
onDelete: Cascade)

  @@unique([pluginManifestId])
  @@index([status])
  @@index([isEnabled])
  @@map("installed_plugins")
}

model PluginAuditLog {
  id               String   @id @default(cuid())
  pluginManifestId String
```

```
  actorUserId      String?
  action           String
  beforeStateJson  Json
  afterStateJson   Json
  createdAt        DateTime @default(now())

  pluginManifest   PluginManifest @relation(fields: [pluginManifestId], references: [id],
onDelete: Cascade)

  @@index([pluginManifestId, createdAt])
  @@map("plugin_audit_logs")
}

enum PluginType {
  pattern
  indicator
  strategy
  risk
  overlay
  signal
  ai
  connector
}

enum PluginStatus {
  enabled
  disabled
  error
  incompatible
  missing_dependency
  update_available
}

enum PluginInstallScope {
  system
  user
}
```

20.2 packages/types 仕様

`packages/types/src/schemas/plugin.schema.ts`

```
import { z } from 'zod';

export const PluginTypeSchema = z.enum([
 'pattern',
 'indicator',
 'strategy',
 'risk',
 'overlay',
```

```
  'signal',
  'ai',
  'connector',
]);

export const PluginStatusSchema = z.enum([
  'enabled',
  'disabled',
  'error',
  'incompatible',
  'missing_dependency',
  'update_available',
]);

export const PluginManifestSchema = z.object({
  id: z.string(),
  slug: z.string(),
  displayName: z.string(),
  version: z.string(),
  descriptionShort: z.string(),
  descriptionLong: z.string(),
  pluginType: PluginTypeSchema,
  authorName: z.string(),
  sourceLabel: z.string(),
  coverImageUrl: z.string().nullable().optional(),
  sourcePreview: z.string().nullable().optional(),
  entryFile: z.string(),
  checksum: z.string(),
  fxdeApiVersion: z.string(),
  fxdeWebVersion: z.string(),
  capabilities: z.array(z.string()),
  permissions: z.array(z.string()),
  dependencies: z.array(z.string()),
  optionalDependencies: z.array(z.string()),
  tags: z.array(z.string()),
  isCore: z.boolean(),
  isSigned: z.boolean(),
  installScope: z.enum(['system', 'user']),
});

export const PluginCardSchema = z.object({
  pluginId: z.string(),
  displayName: z.string(),
  pluginType: PluginTypeSchema,
  summary: z.string(),
  coverImageUrl: z.string().nullable(),
  version: z.string(),
  authorName: z.string(),
  sourceLabel: z.string(),
  isEnabled: z.boolean(),
  status: PluginStatusSchema,
```

```
  compatibilityLabel: z.string(),
  tags: z.array(z.string()),
  isCore: z.boolean(),
  sourcePreviewAvailable: z.boolean(),
});

export const PluginListResponseSchema = z.object({
  items: z.array(PluginCardSchema),
  total: z.number().int().nonnegative(),
});

export const PluginDetailResponseSchema = z.object({
  manifest: PluginManifestSchema,
  installed: z.object({
    id: z.string(),
    isEnabled: z.boolean(),
    status: PluginStatusSchema,
    errorMessage: z.string().nullable(),
    configLocked: z.boolean(),
    installedAt: z.string(),
    enableUpdatedAt: z.string(),
    lastHealthCheckAt: z.string().nullable(),
    lastExecutedAt: z.string().nullable(),
  }),
});

export const PluginSourcePreviewResponseSchema = z.object({
  pluginId: z.string(),
  displayName: z.string(),
  language: z.string(),
  readOnly: z.literal(true),
  content: z.string(),
});

export const TogglePluginResponseSchema = z.object({
  pluginId: z.string(),
  isEnabled: z.boolean(),
  status: PluginStatusSchema,
  enableUpdatedAt: z.string(),
});
```

20.3 Nest API 仕様

Module 構成

```
apps/api/src/modules/plugins/
  plugins.module.ts
  plugins.controller.ts
  plugins.service.ts
  plugins.registry.ts
```

```
  dto/
    get-plugins.query.dto.ts
    plugin-id.param.dto.ts
```

Controller 仕様

```
@Controller('api/v1/plugins')
export class PluginsController {
 @Get()
 getPlugins()

 @Get(':pluginId')
 getPluginDetail()

 @Get(':pluginId/source-preview')
 getSourcePreview()

 @Post(':pluginId/enable')
 @Roles('ADMIN')
 enablePlugin()

 @Post(':pluginId/disable')
 @Roles('ADMIN')
 disablePlugin()

 @Get(':pluginId/audit-logs')
 getAuditLogs()
}
```

Service 振る舞い

- `getPlugins()`
  - manifest + installed を join
  - card view model に整形
  - filter / sort 対応は query param で実装可
- `getPluginDetail(pluginId)`
  - manifest / installed / dependency 情報を返却
- `getSourcePreview(pluginId)`
  - `sourcePreview` を返却
  - ファイルシステムの生読みは MVP では不要

- 返却値に `readOnly: true` を固定

- `enablePlugin(pluginId, actorUserId)`

- 対象取得

- dependency / compatibility を検査
- `isEnabled=true`, `status=enabled`
- audit log 記録

- registry refresh

- `disablePlugin(pluginId, actorUserId)`

- core plugin は拒否可

- `isEnabled=false`, `status=disabled`
- audit log 記録
- registry refresh

Registry 仕様

```
export interface RegisteredPlugin {
  manifest: PluginManifestEntity;
  installed: InstalledPluginEntity;
  moduleRef: unknown | null;
}
```

MVP では実際の動的 import より先に、**DB ベース registry + state 管理** を先行実装する。

20.4 API レスポンス例

`GET /api/v1/plugins`

```
{
  "items": [
    {
      "pluginId": "plg_supply_demand_pro",
      "displayName": "Supply Demand Zones PRO",
      "pluginType": "indicator",
      "summary": "需給ゾーンを可視化する分析プラグイン",
      "coverImageUrl": "/plugin-assets/supply-demand-pro.png",
      "version": "1.0.0",
      "authorName": "msnk",
      "sourceLabel": "Local Signed Plugin",
      "isEnabled": true,
      "status": "enabled",
      "compatibilityLabel": "FXDE v5.1 Compatible",
      "tags": ["zones", "supply-demand", "chart"],
```

```
    "isCore": false,
    "sourcePreviewAvailable": true
  }
 ],
 "total": 1
}
```

`GET /api/v1/plugins/:pluginId/source-preview`

```
{
 "pluginId": "plg_supply_demand_pro",
 "displayName": "Supply Demand Zones PRO",
 "language": "typescript",
 "readOnly": true,
 "content": "export const manifest = ..."
}
```

20.5 React UI 仕様

ディレクトリ

```
apps/web/src/components/strategy/plugins/
  PluginManager.tsx
  PluginToolbar.tsx
  PluginGrid.tsx
  PluginCard.tsx
  PluginDetailDrawer.tsx
  PluginSourceViewer.tsx
  PluginStatusBadge.tsx
  PluginEnableToggle.tsx
  PluginEmptyState.tsx
```

Strategy 統合

- 既存 `Strategy.tsx` に `plugins` タブを追加
- タブの一つとして `PluginManager` をレンダリング
- route は増やさず、既存 page 内 tab state で切り替える

`PluginManager.tsx`

責務:

- 一覧取得
- filter / sort state 管理
- detail open state
- toggle action 呼び出し

`PluginCard.tsx`

表示要素:

- 16:9 cover image
- title
- type badge
- summary
- tags
- version
- author/source label
- status badge
- toggle
- 詳細ボタン

`PluginDetailDrawer.tsx`

表示要素:

- hero image
- long description
- capabilities
- dependencies
- permissions
- source preview viewer
- enable toggle

`PluginSourceViewer.tsx`

仕様:

- read-only 表示専用
- textarea ではなく code block / syntax highlighter を推奨
- 行番号あり
- `Read Only` バッジ
- `Editing is disabled in FXDE` テキスト
- 編集関連 UI は出さない

`PluginEnableToggle.tsx`

仕様:

- `checked={isEnabled}`
- mutate 中は disabled
- 成功時 toast
- failure 時 toast
- optimistic update ではなく refetch or confirmed patch

20.6 hooks 仕様

```
usePlugins(filters)
usePluginDetail(pluginId)
usePluginToggle()
```

- `usePlugins`: list fetch
- `usePluginDetail`: drawer 開閉時に fetch
- `usePluginToggle`: enable / disable API 呼び出し

20.7 シードデータ仕様

最低 2〜3 個のダミープラグインを seed する。

例:

- Supply Demand Zones PRO
- Trend Bias Analyzer
- Session Overlay Pack

すべてに画像 URL、summary、sourcePreview を入れて UI 確認可能にする。

20.8 RBAC 仕様

- 一覧 / 詳細 / source preview: ログイン済ユーザー全体可
- enable / disable: ADMIN のみ
- UI 側でも ADMIN 以外は toggle disabled + tooltip 表示

20.9 テスト観点

Backend

- list 取得
- detail 取得
- source preview 取得
- enable 成功
- disable 成功
- core plugin disable 拒否
- audit log 生成

Frontend

- cards render
- detail opens
- source preview read-only badge 表示
- toggle disabled for non-admin
- toggle success state refresh

---

21. Claudeに渡す資料

Claude には、少なくとも以下をセットで渡す。

必須資料

1. `SPEC_v51_part5.md`
2. Strategy ページ、および `/strategy` 前提の画面整合確認用
3. `SPEC_v51_part10.md`
4. ページ構成、Sidebar、ルーティング正本確認用
5. `FXDE_v51_wireframe_integrated.html`
6. 既存 UI / 情報設計 / Strategy 画面の現物参照用
7. `FXDE Plugin System 完全設計書`
8. 今回作成した Plugin System 全体設計の正本
9. この文書の **19章 / 20章 / 21章**
10. 実装プロンプト、Prisma/Nest/React の具体仕様、渡す資料一覧

できれば渡す資料

1. 現在の最新コード一式
2. 特に `apps/api`, `apps/web`, `packages/types`, `prisma`
3. 直近の監査資料や進捗資料
4. Claude が既存規約を壊さず進めるため

Claude への渡し方

新規会話で、以下の順番を推奨する。

1. まず最新コード一式
2. 次に `SPEC_v51_part5.md`
3. 次に `SPEC_v51_part10.md`
4. 次に `FXDE_v51_wireframe_integrated.html`
5. 次に `FXDE Plugin System 完全設計書`
6. 最後に **19章の「FXDE Plugin System 実装プロンプト」全文**

Claude に必ず明示すること

- `/strategy` の既存確定構成を壊さないこと

- 新規トップレベルページを増やさないこと
- packages/types を必ず正本化すること
- API 契約を先に固めること
- UI から plugin source を編集可能にしないこと
- 既存ルーティング・Sidebar・RBAC を現物確認してから実装すること